

# NUXERA - Detailed Technical Manual

Document generated from NUXERA evidence

|          |                                                               |
|----------|---------------------------------------------------------------|
| Fecha    | 2026-07-29                                                    |
| Paquete  | English                                                       |
| Producto | NUXERA Financial Intelligence                                 |
| Uso      | Revision interna, socio, inversionistas y decision de cutover |

Date: 2026-07-29 Audience: technical team, fractional CTO, integrators, security, DevOps and technical due diligence.

## 1. Overall Architecture

NUXERA keeps the NSD repository while shifting the visible and operating experience to the NUXERA brand. The frontend uses React/Vite and exposes role-based workspaces. The Node/Express backend concentrates NUXERA routes, service contracts, persistence gates, notifications, agents and operational readiness.

Main layers:

- Frontend: public site, dashboard, applicant workspace, funding provider workspace and admin console.
- Backend: protected routes, jurisdiction intelligence services, notification contracts, agent readiness and operational persistence.
- Data: Supabase/SQL as the target for files, events, approvals, documents and traceability.
- AI: OpenAI and Anthropic as primary providers; Kimi/DeepSeek as low-risk secondary providers; NVIDIA as experimental/non-critical.
- Integrations: public sources, private APIs by agreement, transactional email and regulatory registers.

## 2. Frontend

The frontend must preserve three role boundaries:

- Applicant: prepares the file and sees own gaps.
- Funding provider: reviews authorized files, committee material, jurisdiction and risk.
- Admin: operates security, sources, gates, notifications, agents and readiness.

Verified visual scenarios:

- Public home.
- Applicant dashboard.
- Funding provider workspace.
- Admin operations.

Screenshots are stored in docs/nuxera-migration/docs/migration/assets/qa-2026-07-29/.

## 3. Backend and Contracts

Relevant modeled services:

- `nuxeraJurisdictionIntelligenceService`: produces jurisdiction analysis, regulatory sources, scope, limitations and source acquisition planning.
- `nuxeraOperationalPersistenceService`: defines operating events and gates before writing NUXERA state.
- `nuxeraConversationAgentReadinessService`: defines context manifest, agent restrictions and provider policy by risk.
- NUXERA routes: expose readiness, jurisdiction evidence and persistence plans behind authentication.

Technical principle: every sensitive operation must fail closed without token, role and authorization.

LocalStorage demos support visual QA only; they do not prove production authorization.

## 4. Agents and AI Providers

Recommended provider policy:

- OpenAI: reasoning, complex analysis, primary agents and higher-sensitivity compliance tasks.
- Anthropic: document analysis, long explanations, second primary provider and serious fallback.
- Kimi: low-cost provider for low-risk tasks, drafts, preliminary classification, non-binding summaries and internal support.
- DeepSeek: keep behind primary providers and only use for low-risk work when cost/benefit supports it.
- NVIDIA: experimental; presentation or cutover should not depend on it.

The agent should be grounded in:

- File metadata.
- Checklist and requirements.
- Authorized documents.
- Event history.
- Notification state.
- Regulatory sources.
- Country/state/city context.

Restrictions:

- No financing approval.
- No email sending without a gate.
- No cross-user data exposure.
- No invented evidence.
- No treating conditional sources as verified live sources.

## 5. Notifications

Email delivery should move through phases:

- 1 Local dry-run: generate payloads and templates, no delivery.
- 1 Sandbox: send only to approved recipients, no sensitive attachments.

- 1 Limited production: low-sensitivity events with logs and retries.
- 1 Full production: requires legal/privacy approval, signed-off templates and monitoring.

Suggested events:

- File created.
- Missing document.
- Evidence received.
- Status changed.
- Data room invitation.
- SLA approaching breach.
- Human review required.
- Committee decision pending or recorded.

## 6. Data, RLS and Persistence

Before full production, validate:

- Case event tables.
- Notification approval ledger/table.
- Document provenance evidence.
- RLS policies for applicant, funding provider and admin.
- Real or staging-equivalent tokens.
- Applicant cannot access another applicant's file.
- Funding provider only sees authorized data rooms.
- Admin requires privileged role.

## 7. Regulators and Country/City Intelligence

For the Middle East, especially UAE, the platform should classify sources as:

- Public downloadable or searchable.
- Public without stable API.
- Private by agreement.
- Government approval required.
- Unavailable for automation.

Initial UAE sources: UAE PASS, EOCN, SCA/CMA, CBUAE, DFSA, FSRA/ADGM, ADGM Registration Authority, VARA, FTA, National Economic Registry and local economic departments.

For country/state/city intelligence, the engine must separate economic, political, social, regulatory and territorial data, with query date, source and limitation.

## 8. Tests Executed in this Session

- Focused backend: 86/86 tests passed.

- NUXERA frontend: 125/125 tests passed.
- Vite build: passed.
- Playwright screenshots: 4/4 scenarios passed.
- Vercel smoke: passed for main home and production alias.
- Render smoke: 20-second timeout in this run; availability/cold start should be checked before a live API demo.

## 9. Real Technical Pending Work

- 1 Execute RLS phase 2 with real or staging tokens.
- 1 Run non-production SQL persistence rehearsal.
- 1 Test email sandbox with approved recipient.
- 1 Verify Render or move backend to a more predictable runtime before API-dependent demos.
- 1 Define final matrix for private sources and commercial agreements.
- 1 Execute controlled cutover from Nexus to NUXERA.